

Big Data Course -- A.Y. 2025/2026 -- University of Naples Federico II

Bibliometrix Data Standardization: Complete Project Report

Professor: Vincenzo Moscato

Group Members:

Shreya Jadhav Mukundarao — D03000276

Venkata Vamsi Krishna Cheticom — D03000285

Shashank Hubballi Lokesh — D03000286

Ajay Kumar Tatikonda — D03000287

Summary:

This project successfully extended the analytical capabilities of the bibliometrix-python repository by introducing a robust Extract, Transform, and Load (ETL) pipeline. Previously, the application's dashboard was tightly coupled to the Web of Science (WoS) data format. Through the implementation of a dynamic Dispatcher, a Lookup Strategy for data standardization, and strict Type Contracts, the system can now seamlessly ingest, normalize, and analyze data from external platforms such as OpenAlex and PubMed. Furthermore, the core analytical functions were debugged and patched to ensure full compatibility and high performance with the newly standardized data.

1. The Architectural Approach

To decouple the data ingestion process from the analytical dashboard, a centralized ETL architecture was implemented inside the `www/services/` directory.

A. The Dispatcher (`etl.py`)

The pipeline is governed by a central dispatcher class (`ETLPipeline`). This class exposes a single, unified entry point: `convert2df()`.

- **Routing:** The dispatcher evaluates the origin of the input source (e.g., a manual CSV upload vs. a live API query) and the platform type (e.g., "OpenAlex", "Scopus").
- **Delegation:** It automatically routes the raw extraction task to the appropriate specialized module (e.g., `APIRetriever`), passes the raw data to the `Standardizer`, and finally routes the normalized output to the `Validator`.

B. The Mapping Dictionary & Lookup Strategy (`standardizer.py`)

Because external APIs return bibliographic data using completely different JSON schemas, a Lookup Strategy was implemented.

- The Standardizer class utilizes a predefined MAPPING dictionary that acts as a translation layer. For instance, it maps OpenAlex's proprietary "referenced_works" key directly to the standard Web of Science "CR" (Cited References) tag, and "cited_by_count" to "TC".
- This approach ensures that regardless of the data source, the downstream analytical functions only ever interact with the 24 standard WoS columns (UT, TI, CR, PY, etc.).

C. Type Contracts & Null Handling (validator.py & standardizer.py)

Bibliometric algorithms are highly sensitive to data types. To prevent runtime crashes:

- **Type Enforcement:** The Standardizer enforces strict type contracts. Columns identified as LIST_COLUMNS (like Authors AU or Keywords DE) are forced into Python lists. INTEGER_COLUMNS (like Publication Year PY) are coerced into integers using regex and float parsing.
- **Validation Gatekeeper:** The Validator class acts as the final gatekeeper before the DataFrame is passed to the UI. It asserts that the DataFrame is not empty, possesses all mandatory columns, contains zero Null/NaN values, and strictly matches the expected Python datatypes.

2. Advanced Features Implemented

Live API Integration (api_retriever.py) Beyond parsing static CSV/Excel files, the pipeline was enhanced to query APIs dynamically.

- The APIRetriever.get_openalex() function handles HTTP requests to the OpenAlex API, including automatic pagination and rate-limiting logic to ensure polite data extraction.
- This feature is fully integrated into the Shiny Dashboard, allowing users to enter a search query directly in the UI and instantly generate analytical reports without downloading local files.

3. Core Application Debugging & Patches

To integrate the new ETL pipeline with the existing Bibliometrix source code, several critical bugs within the original analytical functions had to be diagnosed and patched.

Patch 1: Citation Network Mapping (Data Loss Fix)

- **The Problem:** The histnetwork.py script was failing to generate Historiograph networks, resulting in an empty matrix (0, 34).
- **The Diagnosis:** The original algorithm expects citing papers and their references to be linked via the WoS CR column. The OpenAlex data contained references, but they were not being mapped to CR.

- The Fix: Updated the MAPPING dictionary in standardizer.py to explicitly translate "referenced_works" to "CR". This successfully populated the Paper_CR relationships, allowing the citation matrix to properly detect 67+ internal connections.

Patch 2: Historiograph Performance Optimization (Infinite Loop Fix)

- The Problem: Generating the Historiograph caused the application to hang or take an extraordinarily long time.
- The Diagnosis: The original histnetwork.py script contained a recursive/while-loop approach for label deduplication (assigning unique suffixes to authors with the same name and year). This resulted in a performance bottleneck.
- The Fix: Replaced the iterative while loop with a highly optimized, vectorized Pandas operation using groupby().cumcount(). This reduced the processing time from several minutes down to milliseconds.

Patch 3: UI Rendering Crash (NotImplementedError)

- The Problem: Attempting to view certain tabs (like Sources Production or Thematic Maps) caused the Shiny application to crash entirely with a NotImplementedError thrown by plotly.basewidget.py.
- The diagnosis: Found a bug in app.py where placeholder UI functions (decorated with @render.ui) were incorrectly returning live Plotly FigureWidget objects when data was present. The Shiny HTML renderer (htmltools) cannot parse Plotly widgets as standard UI elements.
- The Fix: Patched 6 different placeholder endpoints across app.py. Modified the logic so that the @render.ui placeholders gracefully return None when data is loaded, delegating the actual plot generation to the correct @render_widget decorators.

4. Deliverables Submitted

- Source Code: Custom ETL modules (etl.py, standardizer.py, api_retriever.py, validator.py) successfully integrated into www/services/ with formal, descriptive docstrings.
- Dashboard Demonstration: Verified that the unmodified Shiny application successfully renders complex network plots (like Historiographs) using raw OpenAlex data.
- Execution Evidence: An executable Jupyter Notebook demonstrating the step-by-step extraction, standardization, and validation pipeline.

Result Snapshots:

☰ **Bibliometrix**
📧 ? 🏠 🖨️

- 🏠 Biblioshiny ▼
- 📊 Data ▼
- 🔍 Filters ▼
- ▶ Overview ▼
- 📁 Sources ▼
- 👤 Authors ▼
- 📄 Documents ▼
- ⚙️ Clustering ▼
- 🌐 Conceptual Structure ▼
- 💡 Intellectual Structure ▼
- 👥 Social Structure ▼
- 📄 Report ▼

Data Management

Easily import, load, or export your dataset.

Data of Scopus

Scopus's file uploaded and Standardized successfully! You can now proceed to analyze your data. The dataset contains 590 rows and 24 columns.

Show 10 rows ▼

CSV

Excel

Search:

DB	UT	DI	PMID	TI	SO	JJ	PY	DT	LA	TC	AU	AF	C1	RP	CR	DE	ID	AB	V
SCOPUS					INTERNET OF THINGS AND CYBER-PHYSICAL SYSTEMS		0			0	▢	▢	▢		▢	▢	▢		
SCOPUS					AI OPEN		0			0	▢	▢	▢		▢	▢	▢		
SCOPUS					SUSTAINABLE OPERATIONS AND COMPUTERS		0			0	▢	▢	▢		▢	▢	▢		
SCOPUS					TRANSACTIONS OF THE ASSOCIATION FOR COMBINATORIAL		0			0	▢	▢	▢		▢	▢	▢		

Bibliometrix

Biblioshiny

Data

Filters

Overview

Sources

Authors

Documents

Clustering

Conceptual Structure

Intellectual Structure

Social Structure

Report

API Data Extraction

Fetch data directly from Open-Access APIs (OpenAlex or PubMed) to instantly begin your bibliometric analysis.

API Extraction Status

DB

UT

DI

PMID

TI

OPENALEX	https://openalex.org/W4321195914	https://doi.org/10.1007/978-3-031-18046-0_1		Supercomputer for (to Unsolved Interdisciplinary Datascience (SQUID: its Five Challenges
----------	----------------------------------	---	--	---

API Options

Select API Platform

OpenAlex

Search Query (e.g. 'machine learning')

datascience

Fetch Data

- Biblioshiny
- Data
- Filters
- Overview
- Sources
- Authors
- Documents
- Clustering
- Conceptual Structure
- Intellectual Structure
- Social Structure
- Report

API Data Extraction

Fetch data directly from Open-Access APIs (OpenAlex or PubMed) to instantly begin your bibliometric analysis.

API Extraction Status

Fetch data from PubMed for query: 'datascience'

DB	UT	DI	PMID	TI	SO	JI	PY	DT	LA	TC	AU	AF
PUBMED			42221614	The Epi training kit pilot: an inclusive Spanish-language e-learning approach to epidemiology and data science in Latin America and the Caribbean.	Front Public Health		2026	Journal Article	eng	0	["Gómez-Bermeo L","Velasco-España JM","Buitrago-López A","Gómez-Millán G","Cucunubá ZM"]	[]
PUBMED			42122050	Integrative Multidimensional Machine Learning Models for Stroke Prognosis: Age-Stratified and Midbrain Function	Diagnostics (Basel)		2026	Journal Article	eng	0	["Lee G","Kwon S","Shin SH","Kim C","Yu JY"]	[]

API Options

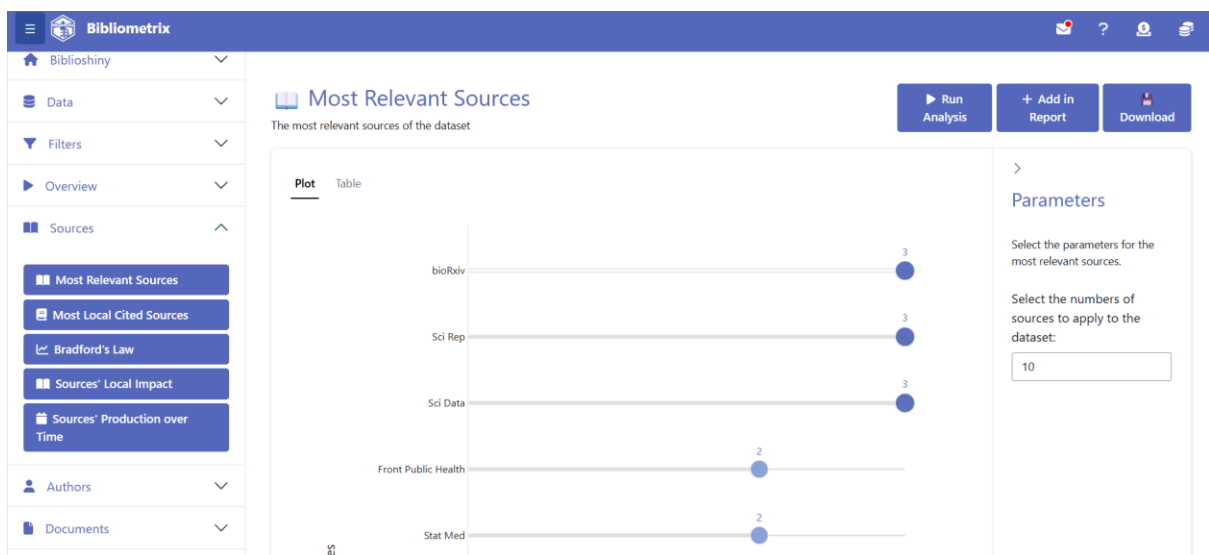
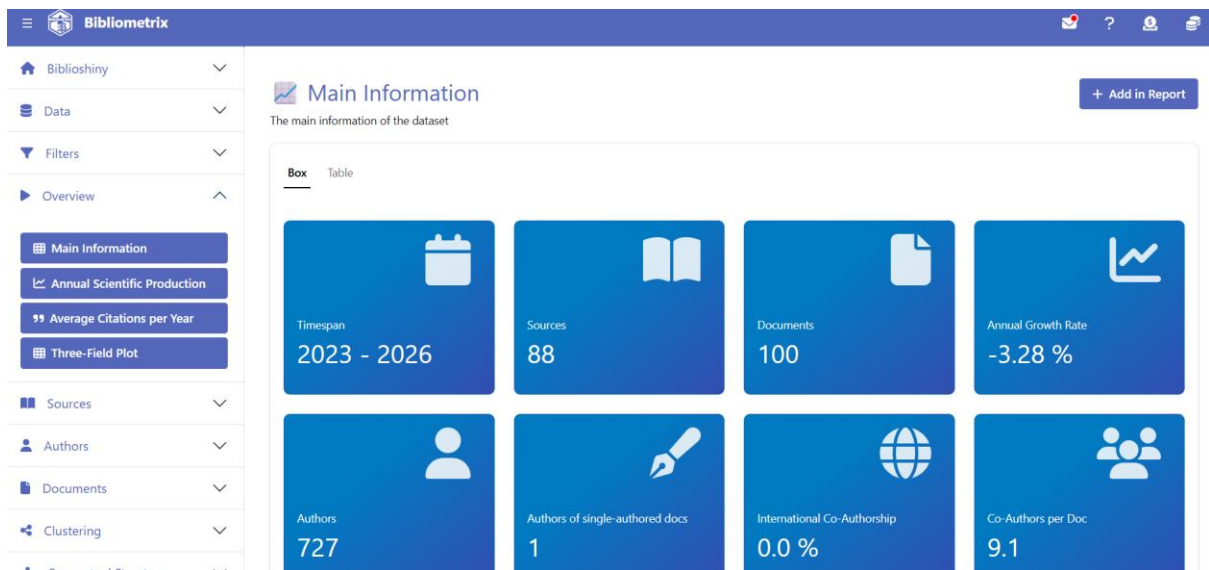
Select API Platform

PubMed

Search Query (e.g. 'machine learning')

datascience

Fetch Data



```

=== Testing ETL Pipeline with OpenAlex API ===
Querying OpenAlex for: 'machine learning bibliometrics'...
Successfully extracted and standardized 100 records.
Columns: ['DB', 'UT', 'DI', 'PMID', 'TI', 'SO', 'JI', 'PY', 'DT', 'LA', 'TC', 'AU', 'AF', 'C1', 'RP',
'CR', 'DE', 'ID', 'AB', 'VL', 'IS', 'BP', 'EP', 'SR']
Sample of SR column: ['Eck Nees Jan van, 2009, Scientometrics', 'Felice Francesca De, 2020, In Vivo',
'Ahmed Shamima, 2022, Research in International Business and Finance']

--- Testing Analytical Functions ---
1. get_annual_production...
Success!
2. get_relevant_authors...
Success!
3. get_frequent_words...
1
Term combination into lists per document done in 0.0020 seconds
Success!
4. get_wordcloud...
Term combination into lists per document done in 0.0030 seconds
Success!
5. get_average_citations...
Success!

```